

Telebirr P2P 中文开发者部署手册

Version 1.0 - Verified 10 July 2026 - Controlled copy

本手册面向第一次接触本项目的开发人员。内容分为本地开发环境、镜像发布、DigitalOcean 生产基础设施、数据服务、API、三个前端、设备 mTLS、OpenClaw、DeepSeek 和 Android APK。

所有命令都从仓库根目录执行，除非步骤中明确写了其他目录。

0. 先确定部署参数

1. 准备五个域名：
2. API：api.example.com
3. 管理后台：admin.example.com
4. 商户后台：merchant.example.com
5. 收银台：checkout.example.com
6. OpenClaw：openclaw.example.com
7. 将本手册中的 <ORG> 替换为 GitHub 组织名。
8. 将 <REPO> 替换为 GitHub 仓库名。
9. 将 <VERSION> 替换为发布版本，例如 0.1.0。
10. 将所有 example.com 替换为真实域名。

1. 项目组成

1. apps/api 是 NestJS API、设备网关和后台任务。
2. apps/admin 是平台管理后台，端口为 4173。
3. apps/merchant 是商户后台，端口为 4174。
4. apps/checkout 是客户存取款页面，端口为 4175。
5. apps/device-agent 是 Android Telebirr Device Agent。
6. packages/contracts 是 API 和设备协议共享类型。
7. integrations/openclaw-telebirr 是 OpenClaw 只读建议插件。
8. infra/docker-compose.yml 只用于本地依赖服务。
9. infra/terraform/digitalocean 用于创建生产基础设施基线。

2. 本地开发环境 - 安装软件

Windows

1. 安装 Git。
2. 安装 Node.js 22。
3. 确认 npm 版本为 10 或更高。
4. 安装 Docker Desktop。
5. 打开 Docker Desktop。
6. 等待左下角显示 Docker Engine 正在运行。
7. 打开 PowerShell。
8. 执行：

```
git --version  
node --version  
npm --version  
docker --version  
docker compose version
```

macOS 或 Linux

1. 安装 Git。
2. 安装 Node.js 22 和 npm 10 或更高版本。
3. 安装 Docker Engine 或 Docker Desktop。
4. 安装 Docker Compose v2。
5. 执行：

```
git --version  
node --version  
npm --version  
docker --version  
docker compose version
```

3. 下载代码

1. 打开终端。
2. 进入准备保存项目的目录。
3. 执行：

```
git clone <GIT_REPOSITORY_URL> telebirr-p2p  
cd telebirr-p2p
```
4. 确认当前目录中存在 package.json。
5. 确认 Node.js 主版本为 22：

```
node --version
```

4. 创建本地配置文件

Windows PowerShell

```
Copy-Item apps/api/.env.example apps/api/.env
Copy-Item apps/admin/.env.example apps/admin/.env
Copy-Item apps/merchant/.env.example apps/merchant/.env
Copy-Item apps/checkout/.env.example apps/checkout/.env
```

macOS 或 Linux

```
cp apps/api/.env.example apps/api/.env
cp apps/admin/.env.example apps/admin/.env
cp apps/merchant/.env.example apps/merchant/.env
cp apps/checkout/.env.example apps/checkout/.env
```

1. 打开 `apps/api/.env`。
2. 找到 `BOOTSTRAP_PLATFORM_ADMIN_EMAIL`。
3. 填写本地管理员邮箱。
4. 找到 `BOOTSTRAP_PLATFORM_ADMIN_PASSWORD`。
5. 填写至少 20 位的本地管理员密码。
6. 找到 `BOOTSTRAP_MERCHANT_OWNER_EMAIL`。
7. 填写本地演示商户邮箱。
8. 找到 `BOOTSTRAP_MERCHANT_OWNER_PASSWORD`。
9. 填写本地演示商户密码。
10. 保存文件。

本地默认连接参数已经写在 `apps/api/.env.example` 中，不需要另外修改 PostgreSQL、Valkey、RabbitMQ 和对象存储地址。

5. 启动本地依赖服务

1. 在仓库根目录执行：

```
docker compose -f infra/docker-compose.yml up -d
```
2. 查看容器状态：

```
docker compose -f infra/docker-compose.yml ps
```
3. 等待 `postgres`、`valkey`、`rabbitmq` 和 `object-storage` 显示健康。
4. RabbitMQ 管理页面地址为 `http://localhost:15672`。
5. 本地 RabbitMQ 用户名为 `telebirr`。
6. 本地 RabbitMQ 密码为 `telebirr`。

6. 安装依赖、迁移数据库并创建本地账号

1. 安装项目依赖：

```
npm ci
```

2. 生成 Prisma Client：

```
npm run prisma:generate
```

3. 执行已有数据库迁移：

```
npm -w @telebirr/api run prisma:deploy
```

4. 创建本地演示数据和登录账号：

```
npm -w @telebirr/api run prisma:seed
```

5. 保存终端输出中的本地演示 API Key。

6. 不要把本地演示 Key 用于 staging 或 production。

7. 启动本地应用

打开四个终端，四个终端都进入仓库根目录。

终端 1 - API

```
npm run dev:api
```

终端 2 - 管理后台

```
npm run dev:admin
```

终端 3 - 商户后台

```
npm run dev:merchant
```

终端 4 - 收银台

```
npm run dev:checkout
```

8. 验证本地部署

1. 打开 <http://localhost:3000/health/live>。
2. 确认返回 `alive`。
3. 打开 <http://localhost:3000/health/ready>。
4. 确认 `database`、`redis` 和 `rabbitmq` 都是 `connected`。
5. 打开 API 文档：<http://localhost:3000/docs>。
6. 打开管理后台：<http://localhost:4173>。
7. 使用 `apps/api/.env` 中的平台管理员账号登录。
8. 打开商户后台：<http://localhost:4174>。
9. 使用本地演示商户账号登录。
10. 打开收银台演示页面：<http://localhost:4175/checkout/demo?token=demo>。

也可以在终端执行：

```
curl http://localhost:3000/health/live
curl http://localhost:3000/health/ready
```

9. 运行测试和构建

1. 执行类型检查：
`npm run typecheck`
2. 执行全部自动化测试：
`npm test`
3. 执行生产构建：
`npm run build`
4. 验证 Docker Compose 配置：
`docker compose -f infra/docker-compose.yml config --quiet`
5. 停止本地依赖时执行：
`docker compose -f infra/docker-compose.yml down`

10. 本地构建生产镜像

1. 构建 API 运行镜像：

```
docker build \  
-f apps/api/Dockerfile \  
--target runtime \  
-t telebirr-api:local .
```

2. 构建数据库迁移镜像：

```
docker build \  
-f apps/api/Dockerfile \  
--target migration \  
-t telebirr-api-migration:local .
```

3. 构建管理后台：

```
docker build \  
-f infra/docker/web.Dockerfile \  
--build-arg APP=admin \  
--build-arg VITE_API_BASE_URL=https://api.example.com/v1 \  
-t telebirr-admin:local .
```

4. 将 APP=admin 分别改为 merchant 和 checkout，构建另外两个前端。

11. 通过 GitHub Actions 发布镜像

1. 打开 GitHub 仓库。

2. 点击 Settings。

3. 点击 Secrets and variables。

4. 点击 Actions。

5. 点击 Variables。

6. 新建变量 PUBLIC_API_BASE_URL。

7. 值填写 https://api.example.com/v1。

8. 合并代码到主分支。

9. 创建版本标签：

```
git tag v<VERSION>  
git push origin v<VERSION>
```

10. 打开 GitHub 的 Actions 页面。

11. 等待 release immutable images 工作流成功。

12. 记录五个镜像的 digest：API、migration、admin、merchant、checkout。

镜像名称格式如下：

```
ghcr.io/<ORG>/<REPO>-api  
ghcr.io/<ORG>/<REPO>-api-migration  
ghcr.io/<ORG>/<REPO>-admin  
ghcr.io/<ORG>/<REPO>-merchant  
ghcr.io/<ORG>/<REPO>-checkout
```

12. 准备 DigitalOcean

1. 登录 DigitalOcean 控制台。
2. 创建一个只用于 Terraform 的 API Token。
3. 创建 Spaces Access Key 和 Secret Key。
4. 在 Settings > Security > SSH Keys 中上传运维 SSH 公钥。
5. 记录 SSH Key fingerprint。
6. 为 API 域名创建 DigitalOcean managed certificate。
7. 为 OpenClaw 域名创建 managed certificate。
8. 准备只允许公司 VPN 或堡垒机访问的 CIDR，例如 203.0.113.10/32。
9. 准备全局唯一的小写 Spaces Bucket 名称。

13. 配置 Terraform

1. 进入 Terraform 目录：
`cd infra/terraform/digitalocean`
2. 复制变量文件：
`cp terraform.tfvars.example terraform.tfvars`
3. 编辑 terraform.tfvars。
4. staging 使用 environment = "staging"。
5. production 使用 environment = "production"。
6. region 保持 fra1。
7. 填写 ssh_fingerprint。
8. 填写 tls_certificate_name。
9. 填写 openclaw_tls_certificate_name。
10. 填写 admin_source_cidrs。
11. 填写 evidence_bucket_name。

生产环境必须在 Terraform 配置中加入团队使用的远程 state backend。不要把生产 state 保存在开发人员电脑上。

14. 执行 Terraform

1. 在终端设置 Terraform 凭证：

```
export TF_VAR_digitalocean_token='<DIGITALOCEAN_TOKEN>'
export TF_VAR_spaces_access_id='<SPACES_ACCESS_ID>'
export TF_VAR_spaces_secret_key='<SPACES_SECRET_KEY>'
```

2. 初始化：

```
terraform init
```

3. 格式化和验证：

```
terraform fmt -check -recursive
terraform validate
```

4. 生成计划：

```
terraform plan -out=plan.tfplan
```

5. 查看计划：

```
terraform show plan.tfplan
```

6. 应用计划：

```
terraform apply plan.tfplan
```

7. 查看输出：

```
terraform output
terraform output -json > deployment-outputs.json
```

8. 将敏感输出写入 Secret Manager。

9. 删除本地 deployment-outputs.json。

Terraform 会创建至少两台 API VM、三台 RabbitMQ VM、一台 OpenClaw VM、托管 PostgreSQL、托管 Valkey、Spaces Bucket 和两个负载均衡器。Terraform 不会自动部署应用容器。

15. 配置 DNS

1. 获取 API Load Balancer IP:

```
terraform output -raw load_balancer_ip
```

2. 获取 OpenClaw Load Balancer IP:

```
terraform output -raw openclaw_load_balancer_ip
```

3. 为 `api.example.com` 创建 A 记录，指向 API Load Balancer IP。
4. 为 `openclaw.example.com` 创建 A 记录，指向 OpenClaw Load Balancer IP。
5. 管理后台、商户后台和收银台域名稍后指向 DigitalOcean App Platform。
6. 等待 DNS 生效。
7. 执行:

```
nslookup api.example.com  
nslookup openclaw.example.com
```

16. 等待服务器初始化

1. 使用 Terraform 输出或 DigitalOcean 控制台找到每台 VM。
2. 通过公司 VPN 或堡垒机 SSH 登录第一台 API VM。
3. 执行:

```
test -f /var/lib/telebirr-provisioned && echo ready  
docker --version  
docker compose version
```

4. 在所有 API、RabbitMQ 和 OpenClaw VM 上重复以上步骤。
5. 只有每台机器都显示 `ready` 后才继续。

17. 配置 RabbitMQ 三节点集群

先通过 Terraform 获取三个 RabbitMQ 私有 IP：

```
terraform output -json rabbitmq_private_ips
```

将三台服务器分别命名为 `rmq1`、`rmq2`、`rmq3`。为三台服务器准备同一个 Erlang Cookie、同一个应用用户名和同一个强密码。

1. 在三台 RabbitMQ VM 和两台 API VM 的私有 DNS 或 `/etc/hosts` 中配置 `rmq1`、`rmq2`、`rmq3`。
2. 每个名称必须解析到对应的 RabbitMQ 私有 IP。
3. 执行 `getent hosts rmq1 rmq2 rmq3` 验证解析。

在每台 RabbitMQ VM 上执行：

```
sudo mkdir -p /opt/telebirr/rabbitmq/data
sudo mkdir -p /opt/telebirr/rabbitmq/certs
sudo chmod 700 /opt/telebirr/rabbitmq/certs
```

1. 把内部 CA 文件放到 `certs/ca.pem`。
2. 把该节点证书放到 `certs/server.pem`。
3. 把该节点私钥放到 `certs/server-key.pem`。
4. 证书必须包含 `rmq1`、`rmq2` 或 `rmq3` 的私有 DNS 名称。
5. 创建 `/opt/telebirr/rabbitmq/rabbitmq.conf`：

```
listeners.tcp = none
listeners.ssl.default = 5671
ssl_options.cacertfile = /etc/rabbitmq/certs/ca.pem
ssl_options.certfile = /etc/rabbitmq/certs/server.pem
ssl_options.keyfile = /etc/rabbitmq/certs/server-key.pem
ssl_options.verify = verify_peer
ssl_options.fail_if_no_peer_cert = false
management.tcp.ip = 127.0.0.1
management.tcp.port = 15672
cluster_partition_handling = pause_minority
default_queue_type = quorum
```

18. 启动 RabbitMQ 节点

在每台 RabbitMQ VM 上设置以下变量，然后执行 Docker 命令。三台机器使用相同 Cookie、用户名和密码，但 NODE_NAME 不同。

```
export NODE_NAME=rmq1
export ERLANG_COOKIE='<SAME_RANDOM_COOKIE>'
export RABBITMQ_USER='telebirr_app'
export RABBITMQ_PASSWORD='<STRONG_PASSWORD>'

sudo docker run -d \
  --name rabbitmq \
  --restart unless-stopped \
  --network host \
  --add-host rmq1:<RMQ1_PRIVATE_IP> \
  --add-host rmq2:<RMQ2_PRIVATE_IP> \
  --add-host rmq3:<RMQ3_PRIVATE_IP> \
  --hostname "$NODE_NAME" \
  -e RABBITMQ_NODENAME="rabbit@$NODE_NAME" \
  -e RABBITMQ_ERLANG_COOKIE="$ERLANG_COOKIE" \
  -e RABBITMQ_DEFAULT_USER="$RABBITMQ_USER" \
  -e RABBITMQ_DEFAULT_PASS="$RABBITMQ_PASSWORD" \
  -e RABBITMQ_DEFAULT_VHOST=/telebirr \
  -v /opt/telebirr/rabbitmq/data:/var/lib/rabbitmq \
  -v /opt/telebirr/rabbitmq/certs:/etc/rabbitmq/certs:ro \
  -v /opt/telebirr/rabbitmq/rabbitmq.conf:/etc/rabbitmq/rabbitmq.conf:ro \
  rabbitmq:4.2.6-management-alpine
```

在 rmq2 和 rmq3 上依次执行：

```
sudo docker exec rabbitmq rabbitmqctl stop_app
sudo docker exec rabbitmq rabbitmqctl reset
sudo docker exec rabbitmq rabbitmqctl join_cluster rabbit@rmq1
sudo docker exec rabbitmq rabbitmqctl start_app
```

在任意节点查看集群：

```
sudo docker exec rabbitmq rabbitmqctl cluster_status
```

确认输出中存在三个节点。

19. 创建生产 Secret

在受控运维机器上执行。每个命令生成一个不同的值。

```
openssl rand -base64 48
```

分别生成：

1. DEVICE_MTLS_PROXY_SECRET
2. WEBHOOK_MASTER_KEY
3. DATA_ENCRYPTION_KEY
4. CHECKOUT_TOKEN_SECRET
5. OPENCLAW_GATEWAY_TOKEN
6. OPENCLAW_TOOL_TOKEN
7. ADMIN_API_TOKEN

生成设备任务签名私钥：

```
openssl genpkey \  
-algorithm EC \  
-pkeyopt ec_paramgen_curve:P-256 \  
-out device-job-signing-key.pem
```

转换为单行环境变量：

```
awk 'NF {printf "%s\\n", $0}' \  
device-job-signing-key.pem
```

将所有值放入 Secret Manager。两个 API 节点必须使用完全相同的设备签名私钥和应用 Secret。

20. 创建 API 生产环境文件

在每台 API VM 创建 `/opt/telebirr/env/api.env`。两台 API VM 使用同一份配置。

```
NODE_ENV=production
PORT=3000
PUBLIC_API_URL=https://api.example.com
CHECKOUT_BASE_URL=https://checkout.example.com
CORS_ALLOWED_ORIGINS=<ADMIN_ORIGIN>,<MERCHANT_ORIGIN>,<CHECKOUT_ORIGIN>

DATABASE_URL=<POSTGRES_URL_WITH_SSLMODE_REQUIRE>
REDIS_URL=redis://USER:PASSWORD@PRIVATE_VALKEY_HOST:25061
RABBITMQ_URL=amqp://telebirr_app:PASSWORD@rmq1:5671/%2Ftelebirr

OBJECT_STORAGE_ENDPOINT=https://fra1.digitaloceanspaces.com
OBJECT_STORAGE_REGION=fra1
OBJECT_STORAGE_BUCKET=YOUR_PRIVATE_BUCKET
OBJECT_STORAGE_ACCESS_KEY=RUNTIME_BUCKET_KEY
OBJECT_STORAGE_SECRET_KEY=RUNTIME_BUCKET_SECRET
OBJECT_STORAGE_FORCE_PATH_STYLE=false

DEVICE_GATEWAY_URL=wss://api.example.com:8443/v1/device/connect
DEVICE_SIGNING_KEY_ID=telebirr-device-v1
DEVICE_JOB_SIGNING_PRIVATE_KEY_PEM=<PKCS8_PEM_WITH_LITERAL_BACKSLASH_N>
DEVICE_MTLS_PROXY_SECRET=...
WEBHOOK_MASTER_KEY=...
DATA_ENCRYPTION_KEY=...
CHECKOUT_TOKEN_SECRET=...
ADMIN_API_TOKEN=...

OPENCLAW_GATEWAY_URL=https://openclaw.example.com
OPENCLAW_GATEWAY_TOKEN=...
OPENCLAW_TOOL_TOKEN=...
OPENCLAW_NAME_REVIEW_AGENT_ID=main

MERCHANT_API_AUTH_ATTEMPTS_PER_MINUTE=60
MERCHANT_API_AUTH_CACHE_SECONDS=300
MERCHANT_API_RATE_LIMIT_PER_MINUTE=2000
ACTIVATION_CODE_TTL_SECONDS=900
EVIDENCE_UPLOAD_BATCH_SIZE=100

TELEGRAM_BOT_TOKEN=
TELEGRAM_CHAT_ID=
TELEGRAM_ALERT_TYPES=*
ALERT_DEDUPE_SECONDS=120
WEBHOOK_BACKLOG_ALERT_THRESHOLD=100
```

1. 三个 Origin 分别填写 admin、merchant 和 checkout 的完整 HTTPS Origin。
2. PostgreSQL URL 必须包含数据库 telebirr、schema=public 和 sslmode=require。
3. URL 中的用户名和密码必须进行 URL 编码。
4. PKCS#8 私钥中的每个换行写成两个字符 \n。
5. RabbitMQ 私有 CA 复制到 API VM 的 `/opt/telebirr/secrets/internal-ca.pem`。
6. API 容器设置 `NODE_EXTRA_CA_CERTS=/run/secrets/internal-ca.pem`。
7. 执行：

```
sudo chmod 600 /opt/telebirr/env/api.env
sudo chmod 600 /opt/telebirr/secrets/internal-ca.pem
```

21. 执行生产数据库迁移

1. 登录第一台 API VM。

2. 登录 GHCR:

```
sudo docker login ghcr.io
```

3. 拉取 migration 镜像 digest:

```
sudo docker pull \
ghcr.io/<ORG>/<REPO>-api-migration@sha256:<DIGEST>
```

4. 只执行一次迁移:

```
sudo docker run --rm \
--network host \
--add-host rmq1:<RMQ1_PRIVATE_IP> \
--add-host rmq2:<RMQ2_PRIVATE_IP> \
--add-host rmq3:<RMQ3_PRIVATE_IP> \
--env-file /opt/telebirr/env/api.env \
-e NODE_EXTRA_CA_CERTS=/run/secrets/internal-ca.pem \
-v /opt/telebirr/secrets/internal-ca.pem:\
/run/secrets/internal-ca.pem:ro \
ghcr.io/<ORG>/<REPO>-api-migration@sha256:<DIGEST>
```

5. 确认退出码为 0。

6. 不要同时在两台 API VM 上执行 migration。

22. 创建第一个平台管理员

1. 临时向 /opt/telebirr/env/api.env 添加:

```
BOOTSTRAP_PLATFORM_ADMIN_EMAIL=admin@example.com
BOOTSTRAP_PLATFORM_ADMIN_NAME=Platform Administrator
BOOTSTRAP_PLATFORM_ADMIN_PASSWORD=<STRONG_20_PLUS_PASSWORD>
```

2. 执行一次 seed:

```
sudo docker run --rm \
--network host \
--env-file /opt/telebirr/env/api.env \
--entrypoint npm \
ghcr.io/<ORG>/<REPO>-api-migration@sha256:<DIGEST> \
-w @telebirr/api run prisma:seed
```

3. 确认输出显示管理员已经创建。

4. 从 api.env 删除 BOOTSTRAP_PLATFORM_ADMIN_PASSWORD。

5. 再次设置文件权限为 600。

23. 启动 API 容器

在每台 API VM 上执行：

```
sudo docker pull \
ghcr.io/<ORG>/<REPO>-api@sha256:<DIGEST>

sudo docker run -d \
--name telebirr-api \
--restart unless-stopped \
--network host \
--add-host rmq1:<RMQ1_PRIVATE_IP> \
--add-host rmq2:<RMQ2_PRIVATE_IP> \
--add-host rmq3:<RMQ3_PRIVATE_IP> \
--env-file /opt/telebirr/env/api.env \
-e NODE_EXTRA_CA_CERTS=/run/secrets/internal-ca.pem \
-v /opt/telebirr/secrets/internal-ca.pem:\
/run/secrets/internal-ca.pem:ro \
ghcr.io/<ORG>/<REPO>-api@sha256:<DIGEST>
```

查看日志：

```
sudo docker logs -f telebirr-api
```

验证本机健康：

```
curl http://127.0.0.1:3000/health/live
curl http://127.0.0.1:3000/health/ready
```

两台 API VM 都必须返回 ready。

24. 配置设备 mTLS 入口

API Load Balancer 的 8443 端口会把 TCP 原样转发到 API VM 的 3443。每台 API VM 都要运行 HAProxy。

1. 准备覆盖 `api.example.com` 的服务器证书和私钥。

2. 将 full chain 和私钥合并为：

```
/opt/telebirr/secrets/device-ingress.pem
```

3. 将允许签发设备证书的客户端 CA 保存为：

```
/opt/telebirr/secrets/device-client-ca.pem
```

4. 把仓库中的模板复制到 API VM：

```
infra/haproxy/device-mtls.cfg.tmpl
```

5. 从 Secret Manager 读取 `DEVICE_MTLS_PROXY_SECRET`。

6. 渲染配置：

```
export DEVICE_MTLS_PROXY_SECRET='<SECRET>'
envsubst < device-mtls.cfg.tmpl \
| sudo tee /opt/telebirr/device-mtls.cfg >/dev/null
unset DEVICE_MTLS_PROXY_SECRET
```

7. 启动 HAProxy：

```
sudo docker run -d \
--name telebirr-device-mtls \
--restart unless-stopped \
--network host \
-v /opt/telebirr/device-mtls.cfg:\
/usr/local/etc/haproxy/haproxy.cfg:ro \
-v /opt/telebirr/secrets/device-ingress.pem:\
/run/secrets/device-ingress.pem:ro \
-v /opt/telebirr/secrets/device-client-ca.pem:\
/run/secrets/device-client-ca.pem:ro \
haproxy:3.1-alpine
```

8. 查看日志：

```
sudo docker logs telebirr-device-mtls
```

9. 没有受信任客户端证书的连接必须失败。

25. 部署三个前端

三个前端镜像监听容器端口 8080。最简单的参考部署方式是 DigitalOcean App Platform。

对 admin、merchant 和 checkout 分别执行以下步骤：

1. 打开 DigitalOcean 控制台。
2. 点击 Create。
3. 点击 App Platform。
4. 选择 Container image。
5. 选择 GitHub Container Registry。
6. 填写 GHCR 用户名和只读 Token。
7. 选择对应镜像。
8. 使用已经记录的固定 digest。
9. 资源类型选择 Web Service。
10. HTTP Port 填写 8080。
11. 健康检查路径填写 /healthz。
12. 部署应用。
13. 在 Settings > Domains 中添加真实域名。

三个镜像分别为：

```
ghcr.io/<ORG>/<REPO>-admin@sha256:<DIGEST>  
ghcr.io/<ORG>/<REPO>-merchant@sha256:<DIGEST>  
ghcr.io/<ORG>/<REPO>-checkout@sha256:<DIGEST>
```

前端的 VITE_API_BASE_URL 在构建镜像时已经固定。修改 API 域名后必须重新构建前端镜像。

26. 部署 OpenClaw Gateway

1. SSH 登录 OpenClaw VM。

2. 克隆官方 OpenClaw 仓库：

```
cd /opt
sudo git clone https://github.com/openclaw/openclaw.git
sudo chown -R "$USER":"$USER" /opt/openclaw
cd /opt/openclaw
```

3. 选择满足本项目插件要求的固定 OpenClaw 版本。

4. 不要在生产环境使用可移动的 `latest` 标签。

5. 设置固定镜像：

```
export OPENCLAW_IMAGE=ghcr.io/openclaw/openclaw:<PINNED_VERSION>
export OPENCLAW_GATEWAY_TOKEN=<OPENCLAW_GATEWAY_TOKEN>
```

6. 运行官方 Docker 安装程序：

```
./scripts/docker/setup.sh
```

7. 按提示完成 Gateway onboarding。

8. Gateway bind 选择可让 Load Balancer 访问的 lan。

9. Gateway 端口保持 18789。

10. 确认 Gateway 使用的 Token 与 API 配置中的 `OPENCLAW_GATEWAY_TOKEN` 完全相同。

11. 确认 Gateway 已启动：

```
docker compose ps
docker compose run --rm openclaw-cli gateway probe
```

官方 Docker 安装说明：

[OpenClaw Docker](#)

27. 配置 DeepSeek

1. 在 OpenClaw VM 的 `/opt/openclaw` 目录执行：

```
docker compose run --rm openclaw-cli \
  plugins install @openclaw/deepseek-provider
```

2. 将 `DEEPSEEK_API_KEY` 写入 OpenClaw 的持久化环境配置。

3. 重新启动 Gateway：

```
docker compose restart openclaw-gateway
```

4. 查看 DeepSeek 模型：

```
docker compose run --rm openclaw-cli \
  models list --provider deepseek
```

5. 默认模型设置为 `deepseek/deepseek-v4-flash`。

6. 不把 `deepseek-v4-pro` 设置为默认模型。

官方 DeepSeek 配置说明：

[OpenClaw DeepSeek provider](#)

28. 安装 Telebirr OpenClaw 插件

在开发机的项目仓库根目录执行：

```
npm ci
npm -w @telebirr/openclaw-tools run build
npm pack -w @telebirr/openclaw-tools
```

1. 将生成的 `.tgz` 文件复制到 OpenClaw VM 的 `/opt/telebirr/plugins`。

2. 在 `/opt/openclaw` 中执行：

```
docker compose cp \
  /opt/telebirr/plugins/telebirr-openclaw-tools-0.1.0.tgz \
  openclaw-gateway:/tmp/telebirr-openclaw-tools.tgz
```

3. 安装插件：

```
docker compose exec openclaw-gateway \
  openclaw plugins install \
  /tmp/telebirr-openclaw-tools.tgz
```

4. 在 OpenClaw Gateway 环境中设置：

```
TELEBIRR_INTERNAL_API_URL=https://api.example.com
TELEBIRR_OPENCLAW_SERVICE_TOKEN=<OPENCLAW_TOOL_TOKEN>
DEEPSEEK_API_KEY=<DEEPSEEK_API_KEY>
```

5. 使用仓库中的 `openclaw.example.json5` 配置允许的工具。

6. 重启 Gateway。

29. 验证 OpenClaw

1. 检查 Gateway:
`docker compose run --rm openclaw-cli gateway probe`
2. 检查设备配对请求:
`docker compose run --rm openclaw-cli devices list --json`
3. 检查 Telebirr 插件是否已启用。
4. 调用只读 fleet summary 工具。
5. 确认工具返回脱敏聚合数据。
6. 确认 OpenClaw 没有 USSD、PIN、账本或付款执行工具。
7. 在 API 日志中确认 `/internal/ai/*` 请求使用独立 Tool Token。

30. 构建 Android Device Agent

1. 安装 JDK 17。
2. 安装 Android SDK 35。
3. 安装 Android Build Tools 35.0.0。
4. 进入 Android 项目：
`cd apps/device-agent`
5. 运行单元测试和 lint：
`./gradlew testDebugUnitTest lintDebug --no-daemon`
6. 构建未签名 release APK：
`./gradlew assembleRelease --no-daemon`
7. 输出文件位于：
`app/build/outputs/apk/release/app-release-unsigned.apk`

31. 签名 Android APK

第一次发布时创建长期保存的 release keystore：

```
keytool -genkeypair \  
-keystore telebirr-agent-release.jks \  
-alias telebirr-agent \  
-keyalg RSA \  
-keysize 4096 \  
-validity 10000
```

每次发布执行：

```
zipalign -p -f 4 \  
app/build/outputs/apk/release/app-release-unsigned.apk \  
telebirr-agent-aligned.apk  
  
apksigner sign \  
--ks telebirr-agent-release.jks \  
--ks-key-alias telebirr-agent \  
--out telebirr-agent-release.apk \  
telebirr-agent-aligned.apk
```

验证签名：

```
apksigner verify --verbose --print-certs \  
telebirr-agent-release.apk  
sha256sum telebirr-agent-release.apk
```

1. 记录版本号、versionCode、证书指纹和 SHA-256。
2. 将签名 APK 上传到 AirDroid 私有应用库。
3. Debug APK 不得上传到生产设备组。

32. 生产上线验证

1. 打开 <https://api.example.com/health/live>。
2. 打开 <https://api.example.com/health/ready>。
3. 打开 <https://api.example.com/docs>。
4. 登录 <https://admin.example.com>。
5. 登录 <https://merchant.example.com>。
6. 确认 <https://checkout.example.com> 可以打开。
7. 确认两台 API VM 都在 Load Balancer 中显示健康。
8. 确认 RabbitMQ cluster_status 中有三个节点。
9. 确认 PostgreSQL PITR 已启用。
10. 确认 Spaces Bucket 为 private，并启用了版本和保留策略。
11. 确认 OpenClaw Gateway 可以被 Android companion 访问。
12. 确认没有客户端证书时，设备 8443 入口拒绝连接。
13. 使用测试 Key 完成一次入金成功模拟。
14. 使用测试 Key 完成一次出金成功模拟。
15. 验证 webhook 签名、重复投递和重放。

33. 日常查看命令

API 日志：

```
sudo docker logs --tail 200 telebirr-api
```

API 健康：

```
curl -fsS http://127.0.0.1:3000/health/ready
```

HAProxy 日志：

```
sudo docker logs --tail 200 telebirr-device-mtls
```

RabbitMQ 集群：

```
sudo docker exec rabbitmq rabbitmqctl cluster_status
```

OpenClaw：

```
docker compose ps  
docker compose run --rm openclaw-cli gateway probe
```

34. 发布新版本

1. 在 staging 执行 `npm run typecheck`。
2. 执行 `npm test`。
3. 执行 `npm run build`。
4. 创建新的 semver Git 标签。
5. 等待 GitHub Actions 发布五个新镜像。
6. 记录全部 digest。
7. 备份数据库并确认 PITR 正常。
8. 运行新版本 migration 镜像一次。
9. 先升级一台 API VM。
10. 等待 `/health/ready` 成功。
11. 观察错误率、队列和数据库锁。
12. 再升级第二台 API VM。
13. 更新三个 App Platform 前端 digest。
14. 最后更新 Android APK 和 OpenClaw 插件。
15. 保留上一版本 digest，直到观察期结束。

35. API 容器升级命令

在一台 API VM 上执行：

```
sudo docker stop telebirr-api
sudo docker rm telebirr-api
sudo docker pull \
  ghcr.io/<ORG>/<REPO>-api@sha256:<NEW_DIGEST>
```

然后使用第 23 节中的 `docker run` 命令启动新 digest。

验证：

```
curl -fsS http://127.0.0.1:3000/health/ready
```

确认第一台恢复健康后，再处理第二台。

36. 备份和恢复

1. DigitalOcean Managed PostgreSQL 启用 PITR。
2. 每天检查最新备份时间。
3. 每季度创建一套隔离数据库并执行恢复演练。
4. 恢复后执行数据库迁移状态检查。
5. 执行账本借贷平衡查询。
6. 验证最近交易、设备、webhook 和审计记录。
7. Spaces 保持 private、versioning 和生命周期策略。

8. 单独备份 OpenClaw 配置、插件版本和 Gateway 配对状态。
9. 单独备份 RabbitMQ 定义和集群配置。
10. Android PIN 不进入任何云备份。

37. 常见部署错误

API 启动时报环境变量错误

1. 查看 `docker logs telebirr-api`。
2. 检查七个生产 Secret 是否不同。
3. 检查每个 Secret 是否至少 32 个字符。
4. 检查 `PUBLIC_API_URL` 和 `CHECKOUT_BASE_URL` 是否为 HTTPS。
5. 检查 `DEVICE_GATEWAY_URL` 是否为 WSS。
6. 检查设备签名私钥是否为 PKCS#8 BEGIN PRIVATE KEY。

`/health/ready` 返回 503

1. 检查 PostgreSQL 连接字符串。
2. 检查 Valkey 私网连接。
3. 检查 RabbitMQ 集群和 TLS CA。
4. 检查 API VM 是否在正确 VPC 和 Firewall Tag 中。

前端无法访问 API

1. 检查构建镜像时的 `VITE_API_BASE_URL`。
2. 检查 API 的 `CORS_ALLOWED_ORIGINS`。
3. 确认 Origin 完全匹配，包括 `https://`。
4. 修改后重新构建并发布前端镜像。

OpenClaw 无法调用 Telebirr 工具

1. 检查 `TELEBIRR_INTERNAL_API_URL` 是否只到域名根路径。
2. 检查 `TELEBIRR_OPENCLAW_SERVICE_TOKEN` 是否等于 API 的 `OPENCLAW_TOOL_TOKEN`。
3. 检查插件是否出现在 OpenClaw plugin 列表中。
4. 检查 Gateway 环境是否包含 DeepSeek API Key。

38. 官方参考

1. [DigitalOcean Terraform Provider](#)
2. [DigitalOcean App Platform 容器镜像部署](#)
3. OpenClaw: [Docker 安装](#)、[Gateway 命令](#)、[DeepSeek provider](#)